

LAB NO#02

Alisha Naushad

(CT-25169)

Question no#01:

/*You are given the heads of two sorted linked lists list1 and list2.

Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists.

Return the head of the merged linked list.

Example: Input: list1 = [1,2,4], list2 = [1,3,4], Output: [1,1,2,3,4,4] */

```
#include<iostream>
```

```
using namespace std;
```

```
struct Node {
```

```
    int data;
```

```
    Node* next;
```

```
    Node(int val) {
```

```
        data = val;
```

```
        next = nullptr;
```

```
    }
```

```
};
```

```
Node* mergelists(Node*list1,Node*list2){
```

```
Node model(0);
```

```
Node*tail=&model;

while(list1!=nullptr && list2!=nullptr){

    if(list1->data<=list2->data){

        tail->next=list1;

        list1=list1->next;

    }

    else{

        tail->next=list2;

        list2=list2->next;

    }

    tail=tail->next;

}

if(list1!=nullptr){

    tail->next=list1;

}

else{

    tail->next=list2;

}

return model.next;

}

void printlist(Node*head){

    while(head!=nullptr){

        cout<<head->data<<"-";

        head=head->next;

    }

}
```

```

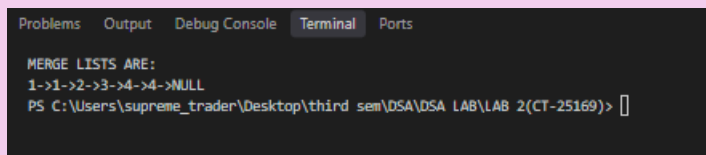
        cout<<"NULL"<<endl;
    }

int main(){
    //list1
    Node*list1=new Node(1);
    list1->next=new Node(2);
    list1->next->next=new Node(4);
    //list2
    Node*list2=new Node(1);
    list2->next=new Node(3);
    list2->next->next=new Node(4);

    Node*result= mergelists(list1,list2);
    cout<<"MERGE LISTS ARE:"<<endl;
    printlist(result);

}

```



```

Problems  Output  Debug Console  Terminal  Ports
MERGE LISTS ARE:
1->1->2->3->4->4->NULL
PS C:\Users\supreme_trader\Desktop\third sem\DSA\DSA LAB\LAB 2(CT-25169)>

```

Question no#02:

Given the head of a sorted linked list, delete all duplicates such that each element appears only once. Return the linked list sorted as well.

```
#include<iostream>

using namespace std;

struct Node {

    int data;

    Node* next;

    Node(int val) {

        data = val;

        next = nullptr;

    }

};

Node* deleteDuplicates(Node* head) {

    if (head == nullptr) {

        return head;

    }

    Node* current = head;

    while (current != nullptr && current->next != nullptr) {

        if (current->data == current->next->data) {

            current->next = current->next->next;

        } else {
```

```
        current = current->next;
    }
}

return head;
}

void printlist(Node* head) {
    while (head != nullptr) {
        cout << head->data << " -> ";
        head = head->next;
    }
    cout << "NULL" << endl;
}

int main() {

    Node* head = new Node(1);
    head->next = new Node(1);
    head->next->next = new Node(2);
    head->next->next->next = new Node(3);
    head->next->next->next->next = new Node(3);

    cout << "Original List:" << endl;
    printlist(head);
}
```

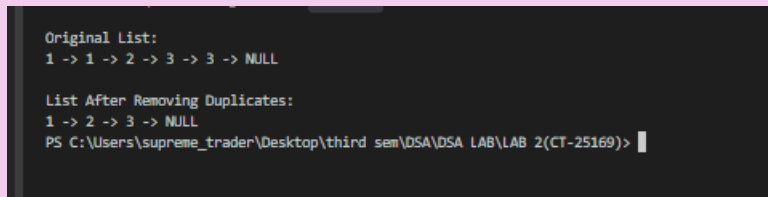
```

head = deleteDuplicates(head);

cout << "\nList After Removing Duplicates:" << endl;
printlist(head);

return 0;
}

```



```

Original List:
1 -> 1 -> 2 -> 3 -> 3 -> NULL

List After Removing Duplicates:
1 -> 2 -> 3 -> NULL
PS C:\Users\supreme_trader\Desktop\third sem\DSA\DSA LAB\LAB 2(CT-25169)>

```

Question no#03:

/*Given the head of a linked list, return the list after sorting it in ascending order. Use mergesort. */

```

#include <iostream>

using namespace std;

```

```

struct Node {

    int data;

    Node* next;

    Node(int val) {

        data = val;

        next = nullptr;

    }
}

```

```
};
```

```
Node* merge(Node* l1, Node* l2) {
```

```
    if (l1 == nullptr) return l2;
```

```
    if (l2 == nullptr) return l1;
```

```
    if (l1->data < l2->data) {
```

```
        l1->next = merge(l1->next, l2);
```

```
        return l1;
```

```
    } else {
```

```
        l2->next = merge(l1, l2->next);
```

```
        return l2;
```

```
    }
```

```
}
```

```
Node* getMid(Node* head) {
```

```
    Node* slow = head;
```

```
    Node* fast = head->next;
```

```
    while (fast != nullptr && fast->next != nullptr) {
```

```
        slow = slow->next;
```

```
        fast = fast->next->next;
```

```
    }
```

```
    return slow;
```

```
}
```

```
Node* mergeSort(Node* head) {
```

```
if (head == nullptr || head->next == nullptr) {  
    return head;  
}
```

```
Node* mid = getMid(head);
```

```
Node* right = mid->next;
```

```
mid->next = nullptr;
```

```
Node* leftSorted = mergeSort(head);
```

```
Node* rightSorted = mergeSort(right);
```

```
return merge(leftSorted, rightSorted);  
}
```

```
void printList(Node* head) {  
    while (head != nullptr) {  
        cout << head->data;  
        if (head->next != nullptr) cout << " -> ";  
        head = head->next;  
    }  
    cout << endl;  
}
```

```
int main() {  
    Node* head = new Node(4);  
    head->next = new Node(2);
```

```

head->next->next = new Node(1);
head->next->next->next = new Node(3);

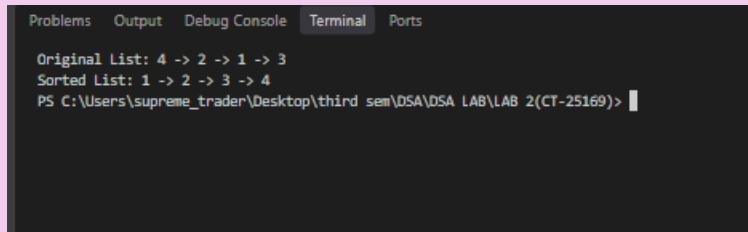
cout << "Original List: ";
printList(head);

head = mergeSort(head);

cout << "Sorted List: ";
printList(head);

return 0;
}

```



```

Problems Output Debug Console Terminal Ports
Original List: 4 -> 2 -> 1 -> 3
Sorted List: 1 -> 2 -> 3 -> 4
PS C:\Users\supreme_trader\Desktop\third sem\DSA\DSA LAB\LAB 2(CY-25169)>

```

Question no#04:

/*Given the head of a singly linked list, return true if it is a palindrome or false otherwise. */

```

#include <iostream>

using namespace std;

struct Node {
    int data;

```

```
Node* next;

Node(int val) {
    data = val;
    next = nullptr;
}

};
```

```
Node* reverseList(Node* head) {
    Node* prev = nullptr;
    Node* curr = head;
    while (curr != nullptr) {
        Node* nextTemp = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextTemp;
    }
    return prev;
}
```

```
bool isPalindrome(Node* head) {
    if (head == nullptr || head->next == nullptr) return true;

    Node* slow = head;
    Node* fast = head;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
```

```
        fast = fast->next->next;
    }

    Node* secondHalf = reverseList(slow);
    Node* firstHalf = head;

    while (secondHalf != nullptr) {
        if (firstHalf->data != secondHalf->data) {
            return false;
        }
        firstHalf = firstHalf->next;
        secondHalf = secondHalf->next;
    }

    return true;
}

void printList(Node* head) {
    while (head != nullptr) {
        cout << head->data;
        if (head->next != nullptr) cout << " -> ";
        head = head->next;
    }
    cout << endl;
}
```

```
int main() {  
  
    Node* head = new Node(1);  
    head->next = new Node(2);  
    head->next->next = new Node(2);  
    head->next->next->next = new Node(1);  
  
    cout << "Input List: ";  
    printList(head);  
  
    if (isPalindrome(head)) {  
        cout << "Output: true" << endl;  
    } else {  
        cout << "Output: false" << endl;  
    }  
  
    return 0;  
}
```

Problems Output Debug Console Terminal Ports

Input List: 1 -> 2 -> 2 -> 1

Output: true

PS C:\Users\supreme_trader\Desktop\third sem\DSA\DSA LAB\LAB 2(CT-25169)> |

Question no#05:

/*Implement class of a Circular Queue using a Linked List. */

```
#include <iostream>
```

```
using namespace std;
```

```
struct Node {
```

```
    int data;
```

```
    Node* next;
```

```
    Node(int val) {
```

```
        data = val;
```

```
        next = nullptr;
```

```
    }
```

```
};
```

```
class CircularQueue {
```

```
private:
```

```
    Node* front;
```

```
    Node* rear;
```

```
public:
```

```
    CircularQueue() {
```

```
        front = nullptr;
```

```
        rear = nullptr;
```

```
    }
```

```
    bool isEmpty() {
```

```
    return front == nullptr;
}
```

```
void enqueue(int val) {
    Node* newNode = new Node(val);
    if (isEmpty()) {
        front = rear = newNode;
        rear->next = front;
    } else {
        rear->next = newNode;
        rear = newNode;
        rear->next = front;
    }
    cout << val << " enqueued" << endl;
}
```

```
void dequeue() {
    if (isEmpty()) {
        cout << "Queue is Empty!" << endl;
        return;
    }
```

```
    if (front == rear) {
        cout << front->data << " dequeued" << endl;
        delete front;
        front = rear = nullptr;
```

```
    } else {  
        Node* temp = front;  
        cout << temp->data << " dequeued" << endl;  
        front = front->next;  
        rear->next = front;  
        delete temp;  
    }  
}
```

```
void getFront() {  
    if (isEmpty()) {  
        cout << "Queue is Empty!" << endl;  
    } else {  
        cout << "Front element: " << front->data << endl;  
    }  
}
```

```
void display() {  
    if (isEmpty()) {  
        cout << "Queue is Empty!" << endl;  
        return;  
    }  
}
```

```
Node* temp = front;  
cout << "Queue: ";  
do {
```

```
        cout << temp->data << " ";  
        temp = temp->next;  
    } while (temp != front);  
    cout << endl;  
}  
};
```

```
int main() {  
    CircularQueue q;  
  
    q.enqueue(10);  
    q.enqueue(20);  
    q.enqueue(30);  
  
    q.display();  
  
    q.getFront();  
  
    q.dequeue();  
    q.display();  
  
    q.enqueue(40);  
    q.display();  
  
    return 0;  
}
```

```
Problems Output Debug Console Terminal Ports
10 enqueued
20 enqueued
30 enqueued
Queue: 10 20 30
Front element: 10
10 dequeued
Queue: 20 30
40 enqueued
Queue: 20 30 40
PS C:\Users\supreme_trader\Desktop\third sem\DSA\DSA LAB\LAB 2(CT-25169)> |
```

Question no#06:

/*Implement class of a Stack using a Linked List. */

```
#include <iostream>
```

```
using namespace std;
```

```
struct Node {
```

```
    int data;
```

```
    Node* next;
```

```
    Node(int val) {
```

```
        data = val;
```

```
        next = nullptr;
```

```
    }
```

```
};
```

```
class Stack {
```

```
private:
```

```
    Node* top;
```

public:

```
Stack() {
```

```
    top = nullptr;
```

```
}
```

```
bool isEmpty() {
```

```
    return top == nullptr;
```

```
}
```

```
void push(int val) {
```

```
    Node* newNode = new Node(val);
```

```
    newNode->next = top;
```

```
    top = newNode;
```

```
    cout << val << " pushed into stack" << endl;
```

```
}
```

```
void pop() {
```

```
    if (isEmpty()) {
```

```
        cout << "Stack Underflow!" << endl;
```

```
        return;
```

```
    }
```

```
    Node* temp = top;
```

```
    cout << temp->data << " popped from stack" << endl;
```

```
    top = top->next;
```

```
    delete temp;
```

```
}
```

```
int peek() {  
    if (isEmpty()) {  
        cout << "Stack is Empty!" << endl;  
        return -1;  
    }  
    return top->data;  
}
```

```
void display() {  
    if (isEmpty()) {  
        cout << "Stack is Empty!" << endl;  
        return;  
    }  
    Node* temp = top;  
    cout << "Stack elements: ";  
    while (temp != nullptr) {  
        cout << temp->data << " ";  
        temp = temp->next;  
    }  
    cout << endl;  
}
```

```
~Stack() {  
    while (!isEmpty()) {  
        Node* temp = top;
```

```
        top = top->next;
        delete temp;
    }
}
};
```

```
int main() {
    Stack s;

    s.push(10);
    s.push(20);
    s.push(30);

    s.display();

    cout << "Top element: " << s.peek() << endl;

    s.pop();
    s.display();

    return 0;
}
```

Problems Output Debug Console **Terminal** Ports

```
10 pushed into stack
20 pushed into stack
30 pushed into stack
Stack elements: 30 20 10
Top element: 30
30 popped from stack
Stack elements: 20 10
PS C:\Users\supreme_trader\Desktop\third sem\DSA\DSA LAB\LAB 2(CT-25169)> |
```